COMPUTER SCIENCE STUDY SERIES • DATA STRUCTURES

LINKED LISTS: THEORY, ALGORITHMS & IMPLEMENTATION

Topic: Singly & Doubly Linked Lists

Prerequisites: Pointers, Memory Allocation & Arrays

1. INTRODUCTION: ARRAYS VS. DYNAMIC LINKED LISTS

In computer science, storing collections of data requires choosing between contiguous and non-contiguous memory allocations. While **Arrays** occupy fixed, adjacent memory slots (like seats reserved on a bus), a **Linked List** allocates memory dynamically anywhere in RAM, connecting elements via **pointers**.

2. REAL-WORLD CONCEPTUAL ANALOGIES

Analogy 1: The Express Train ("Bogie & Silsila"): Each bogie holds passengers (Data) and is linked via a coupling joint (Pointer) to the next bogie. The train's Engine acts as the `Head` pointer, and the Guard's Van at the end points to `NULL`.

Analogy 2: The Treasure Hunt: Each clue chit contains a message (Data) and instructions/coordinates to find the next hidden clue (Pointer). You keep moving from clue to clue until you reach a clue marked "END" (`NULL`).

3. ANATOMY OF A NODE & VISUAL ARCHITECTURE

A Linked List consists of individual building blocks called **Nodes**. Each node contains two essential components:

- **Data Field:** Stores the actual payload (integers, strings, objects).
- **Next Pointer (Link):** Stores the memory address of the subsequent node in the list.

SINGLY LINKED LIST MEMORY LAYOUT

4. THEORETICAL & TIME COMPLEXITY COMPARISON

PROPERTY / OPERATION	ARRAY	SINGLY LINKED LIST	DOUBLY LINKED LIST
Memory Layout	Contiguous block	Dynamic / Scattered	Dynamic / Scattered
Accessing Element (Index k)	$O(1)$ (Random Access)	$O(n)$ (Sequential Traversal)	$O(n)$ (Sequential Traversal)
Insertion / Deletion at Start	$O(n)$ (Shifting required)	$O(1)$ (Re-point Head)	$O(1)$ (Re-point Head & Prev)
Insertion / Deletion at End	$O(1)$ (If space available)	$O(n)$ (O(1) if Tail pointer kept)	$O(1)$ (With Tail pointer)
Memory Overhead per Element	None (Data only)	1 Pointer per node	2 Pointers per node (Prev & Next)

IMPLEMENTATION GUIDE & CODE REFERENCE

CODE EXAMPLES: PYTHON & C++ IMPLEMENTATIONS

1. FULL PYTHON IMPLEMENTATION (SINGLY LINKED LIST)

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None  # Pointer initialized to NULL

class LinkedList:
    def __init__(self):
        self.head = None

    # Insert a new node at the beginning: O(1)
    def insert_at_head(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    # Insert at the end: O(n)
    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node

    # Delete first node matching target key: O(n)
    def delete_value(self, key):
        curr = self.head
        if curr and curr.data == key:
            self.head = curr.next
            curr = None
            return
        prev = None
        while curr and curr.data != key:
            prev = curr
            curr = curr.next
        if curr is None: return # Key not found
        prev.next = curr.next
        curr = None

    # Traverse and print list: O(n)
    def display(self):
        elements = []
        curr = self.head
        while curr:
            elements.append(str(curr.data))
            curr = curr.next
        print(" -> ".join(elements) + " -> NULL")
```

2. C++ DYNAMIC MEMORY POINTER IMPLEMENTATION

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
```

```
};

void insertAtBeginning(Node*& head, int val) {
    Node* newNode = new Node(val);
    newNode->next = head;
    head = newNode;
}

void printList(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL
";
}
```

STUDENT WORKSHEET & COMPREHENSIVE LAB EXERCISES

PRACTICE WORKSHEET: POINTER TRACING & ALGORITHMS

QUESTION 1: CONCEPTUAL ANALOGY APPLICATION

Using the **Express Train (Bogie & Silsila)** analogy, explain what happens when a node in the middle of a Linked List is deleted. How does this differ from deleting an element from a standard Array in terms of memory operations?

QUESTION 2: POINTER TRACING & STATE DIAGRAMMING

Given the following initial Linked List: `[Head]` → `(12)` → `(35)` → `(78)` → `(99)` → `NULL`

- **Q2a:** What is the value of `Head.next.next.data`? _ _ _ _ _ _ _
- **Q2b:** Trace the pointers if we execute: `Head.next = Head.next.next`. Draw the resulting list below and state which node becomes orphaned (unreachable):

QUESTION 3: CODE OUTPUT PREDICTION & TRACING CHALLENGE

Examine the following Python function and predict its exact printed output for the given list:

```
def mystery_function(head):
    curr = head
    while curr and curr.next:
        curr.data = curr.data + curr.next.data
        curr = curr.next.next
# Initial List: 5 -> 10 -> 15 -> 20 -> 25 -> NULL
```

Predicted List Output after execution:

QUESTION 4: ALGORITHM COMPLETION (FILL IN THE MISSING CODE)

Complete the missing lines of code below to implement a function that **reverses** a Singly Linked List in-place:

```
def reverse_linked_list(head):
    prev = None
    curr = head
    while curr is not None:
        next_temp = curr.next       # Step 1: Save next node
        curr.next = _______________  # Step 2: [MISSING LINE A] Reverse pointer
        prev = _______________       # Step 3: [MISSING LINE B] Move prev forward
        curr = next_temp            # Step 4: Move curr forward
    return _______________          # Step 5: [MISSING LINE C] Return new head
```

Write missing lines:

Line A: `curr.next =` _ _ _ _ _ _ _ | Line B: `prev =` _ _ _ _ _ _ _ | Line C: `return` _ _ _ _ _ _ _

EXAM PREPARATION & SELF-EVALUATION

MASTERY ASSESSMENT & QUICK REFERENCE

SECTION A: MULTIPLE CHOICE ASSESSMENT

Q1. What is the worst-case time complexity to search for a specific value in an unsorted Singly Linked List of size n?

[A] $O(1)$ [B] $O(\log n)$ [C] $O(n)$ [D] $O(n^2)$

Q2. In a Doubly Linked List, how many pointers does each node contain?

[A] 1 (Next) [B] 2 (Prev & Next) [C] 3 (Head, Prev, Next) [D] Depends on list size

Q3. What happens if you lose the reference pointer to the Head node of a Singly Linked List?

[A] The list automatically reverses itself

[B] You can recover the list starting from the Tail node

[C] The entire list becomes unreferenced in memory (Garbage Collection / Memory Leak)

SECTION B: SHORT ANSWER & MEMORY LEAK ANALYSIS

Q4. Explain what a "Memory Leak" is in C/C++ when working with dynamic nodes using `new` and `delete`. How do you prevent it when deleting a list?

Q5. Write pseudo-code or Python code to detect if a Linked List contains a cycle/loop (Floyd's Cycle Finding / Slow & Fast Pointer Algorithm).

SUMMARY CHECKLIST: KEY CONCEPTS MASTERED

1. **Node Construction:** Understanding Data vs. Next Pointer compartments.
2. **Time Complexity Mechanics:** Fast $O(1)$ insertions at Head vs $O(n)$ lookup traversal.
3. **Code Proficiency:** Implementing Node instantiation, insertion, deletion, and reversing algorithms.
4. **Pointer Safety:** Preventing lost references, orphaned nodes, and memory leaks.